Getting Started with Mu2e Batch in MC-1

Slurm on the MC-1 trigger worker cluster

2026-10-08

The MC-1 cluster runs **Slurm 26.05** on the Mu2e trigger workers `mu2e-mc1-worker-01` … `-18`. Worker-01 is the **submit node**. Workers 02–18 (17 nodes) run the jobs.

Submit node	`mu2e-mc1-worker-01.fnal.gov` (runs no jobs)
Compute nodes	`mu2e-mc1-worker-[02-18]`, partition `mc1` (the default)
Workers 02–16	16 CPUs (2 × 8 cores), about 60 GB of memory for jobs
Workers 17–18	32 CPUs (16 cores × 2 threads), about 60 GB of memory for jobs
Time limit	none set; always give `--time` anyway

Every command in this guide was run on the cluster on 2026-10-08.

1. Cluster access via ssh

The workers sit inside the DAQ enclave and have no direct login. You reach them through the gateway `g2gateway01.fnal.gov`, using your Fermilab Kerberos ticket. There is no password or ssh-key login.

Step 1: get a ticket on your own machine.

```
kinit <username>@FNAL.GOV
klist                       # check: "Default principal: <username>@FNAL.GOV"
```

Step 2: log in to the submit node through the gateway.

```
ssh -J mu2eshift@g2gateway01.fnal.gov mu2ebatch@mu2e-mc1-worker-01.fnal.gov
```

`-J` (ProxyJump) logs in to the gateway as `mu2eshift`, then on to worker-01. Use `mu2eshift@` or `mu2edaq@` instead of `mu2ebatch@` to land in those accounts.

Optional: a shortcut in `~/.ssh/config`. With this block, `ssh mc1` is all you need to type:

```
Host mc1
    HostName mu2e-mc1-worker-01.fnal.gov
    User mu2ebatch
    ProxyJump mu2eshift@g2gateway01.fnal.gov
    GSSAPIAuthentication yes
    GSSAPIDelegateCredentials yes
```

If it fails:

Symptom	Cause	Fix
`Permission denied (gssapi-keyex,gssapi-with-mic)`	No ticket, an expired one, or your principal is not authorised for that account	Run `kinit` again. If it still fails, ask a DAQ expert to add your principal.
Hangs, then times out	Wrong gateway	Use `g2gateway01`; `g2gateway02` does not accept these logins.
`xauth: … .Xauthority does not exist`	Harmless warning on the first login	Ignore it.

Windows 11: the built-in OpenSSH client supports `-J`. Kerberos is simplest from WSL2: run `kinit` and `ssh` inside the Linux shell.

2. Login accounts

Batch work runs under shared Mu2e accounts, not personal ones. Your Kerberos principal decides which accounts you may log in to: it must be listed in that account's `~/.k5login`.

Account	Use it for	Can submit jobs
`mu2ebatch`	Batch production and analysis jobs. **Start here.**	yes
`mu2eshift`	Shifter account; DAQ shift tools	yes
`mu2edaq`	DAQ expert account	yes

- **Same account on every node.** The UID is the same everywhere, and so is the environment: `~/.bashrc`, the aliases and the `~/bin` tools from `daq-shifter-tools`.
- **Who may log in:** `mu2ebatch` accepts everyone authorised for `mu2eshift` or `mu2edaq`.
- **Only these three can submit.** Slurm accounting rejects any other user.
- **Known issue (`mu2ebatch`):** an interactive login prints a Kerberos `Fail` line and leaves you without a ticket, because `mu2ebatch` has no group keytab yet. Jobs are not affected. You only need a ticket to ssh onward from the node; run `kinit <username>@FNAL.GOV` there if you do.

3. Available disk areas

Every disk area is local to one node. There is no shared filesystem yet. A file written on worker-05 exists only on worker-05.

Path	Where	Size	Use it for
`/home/<account>`	every node, separate copy each	15 GB per node, shared by all accounts	Login files and small scripts. Not for data.
`/scratch`	every node	100 GB	Temporary files while a job runs
`/data-local`	every node	500 GB	Job output until you collect it
`/tmp`	every node	50 GB	Small temporary files
`/nfshome`	worker-17 only	2.9 TB	Shared home area **once NFS is enabled**
`/nfsdata`	worker-18 only	2.9 TB	Shared data area **once NFS is enabled**

- **Make your own directory.** `/scratch` and `/data-local` are writable by everyone (sticky bit), so work under `/scratch/$USER` and `/data-local/$USER`. To create it on every node: `srun -N17 --chdir=/tmp mkdir -p /data-local/$USER /scratch/$USER`
- **Nothing is backed up or cleaned automatically.** Collect your results and delete what you no longer need.
- **Shared areas are coming.** `/nfshome` and `/nfsdata` will be mounted on every worker once NFS service is approved. Until then, use the collection methods in section 5.

---

4. Submission examples

All commands run on the submit node (worker-01).

Check the cluster

```
sinfo                          # mc1* up, 17 nodes idle
sinfo -N -l                    # per-node CPUs, memory and state
srun -N1 hostname -s           # run one command on any node
srun -N17 --chdir=/tmp hostname -s | sort      # one task on every node
```

A batch job

Save as `hello.sbatch`:

```
#!/bin/bash
#SBATCH --job-name=hello
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --time=00:10:00
#SBATCH --chdir=/data-local
#SBATCH --output=/data-local/%u/slurm-%x-%j.out

echo "job $SLURM_JOB_ID on $(hostname -s) with $SLURM_CPUS_PER_TASK CPUs"
srun sleep 5
```

```
srun -N17 --chdir=/tmp mkdir -p /data-local/$USER   # once: output dir on every node
sbatch hello.sbatch                                 # → Submitted batch job 56
squeue -u $USER                                     # pending / running
sacct -j 56 -X -o JobID,JobName,State,Elapsed,NodeList%20,ExitCode
```

`%u` is your user name, `%x` the job name and `%j` the job id. The output directory must already exist on the node that runs the job. The `mkdir` line above creates it everywhere.

Many independent tasks (job array)

```
#!/bin/bash
#SBATCH --job-name=sweep
#SBATCH --array=0-99%34              # 100 tasks, at most 34 running at once
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=2G
#SBATCH --time=01:00:00
#SBATCH --chdir=/scratch
#SBATCH --output=/data-local/%u/sweep-%A_%a.out

./my_program --index "$SLURM_ARRAY_TASK_ID" \
             --out /data-local/$USER/sweep/result_$SLURM_ARRAY_TASK_ID.root
```

Multi-node job, with its input copied to every node

```
#!/bin/bash
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=8
#SBATCH --time=02:00:00
#SBATCH --output=/data-local/%u/multi-%j.out

sbcast /data-local/$USER/input.root /scratch/$USER/input.root   # copy to all 4 nodes
srun my_program /scratch/$USER/input.root
```

Interactive session

```
salloc -N1 -c8 --mem=16G --time=1:00:00   # reserve a slot on one node
srun --pty bash                           # get a shell on that node
exit                                      # leave the shell, then exit again to release the
        slot
```

Watching and controlling jobs

Task	Command
My jobs	`squeue -u $USER`
Why is my job pending?	`squeue -j <id> -o "%i %T %r"`
Full job details	`scontrol show job <id>`
Cancel one job, or all of mine	`scancel <id>`, `scancel -u $USER`
Today's history	`sacct -S today -X -o JobID,JobName,State,Elapsed,MaxRSS,NodeList%20`

Limits that will bite

- **Memory is enforced.** A job that uses more than it asked for is killed, and `sacct` shows `OUT_OF_MEMORY`. Without a request, a job gets 2000 MB per CPU.
- **CPUs are handed out as whole cores.** On workers 17–18, `-c1` still takes one core, which is two hardware threads.
- **No time limit is enforced, but set `--time` anyway.** Short jobs with an accurate `--time` start sooner, because the scheduler can fit them into gaps.

---

5. Getting data back

A job's output sits on the compute node that ran it. Getting it to your laptop takes two steps: **node → worker-01**, then **worker-01 → your machine**.

Step 1: from the compute node to worker-01

Find the node:

```
node=$(sacct -j <id> -X -n -P -o NodeList)       # e.g. mu2e-mc1-worker-05
```

`-P` matters: without it, `sacct` cuts the node name to 15 characters.

Look at a file without copying it:

```
srun -w $node --chdir=/tmp cat /data-local/$USER/slurm-hello-<id>.out
```

Copy a whole directory back by streaming it through Slurm. This works from every account:

```
mkdir -p /data-local/$USER/collected
srun -w $node --chdir=/tmp tar -C /data-local/$USER -cf - myrun \
    | tar -C /data-local/$USER/collected -xf -
```

Collect from every node after an array or multi-node job:

```
for n in $(sinfo -h -N -o %N | sort -u); do
  srun -w $n --chdir=/tmp tar -C /data-local/$USER -cf - sweep 2>/dev/null \
    | tar -C /data-local/$USER/collected -xf -
done
```

`mu2eshift` and `mu2edaq` can also `scp`/`ssh` directly between workers, because their login gives them a group ticket. `mu2ebatch` can do the same after running `kinit` in that session.

Step 2: from worker-01 to your machine

Run these on your own computer, with a valid ticket:

```
scp -J mu2eshift@g2gateway01.fnal.gov \
    mu2ebatch@mu2e-mc1-worker-01.fnal.gov:/data-local/mu2ebatch/collected/myrun/out.root .

rsync -av -e "ssh -J mu2eshift@g2gateway01.fnal.gov" \
    mu2ebatch@mu2e-mc1-worker-01.fnal.gov:/data-local/mu2ebatch/collected/ ./collected/
```

With the `mc1` shortcut from section 1, this shortens to `scp mc1:/data-local/mu2ebatch/collected/myrun/out.root .`

Clean up once you have the data: `srun -N17 --chdir=/tmp rm -rf /data-local/$USER/myrun`

Getting help

- **Cluster or account problems:** Mu2e DAQ experts. Include the job id and the output of `scontrol show job <id>`.
- **Configuration:** maintained in `Mu2e/mu2edaq-batch-queues`, including `docs/USAGE.md` and the operator documentation.